ChatGPT

Executive Summary

SENTINEL (AI Payment Trust & Recovery) is a proposed end-to-end platform to automate PayPal dispute resolution, fraud detection, and chargeback prevention using AI. It integrates PayPal's Orders v2/Payments v2 APIs (for initiating payments), **Webhooks** (to receive real-time payment and dispute events), and the **Disputes API** (for programmatic handling of chargebacks)【13†L80-L89】【44†L72-L75】. On top of this, SENTINEL leverages machine-learning (ML) models (e.g. XGBoost, LightGBM) trained on public fraud datasets (like the IEEE-CIS Fraud dataset and synthetic "PaySim" data)【41†L33-L41】【35†L224-L232】 to flag suspicious transactions. It also builds LLM-based investigation agents (using frameworks like LangChain or LlamaIndex) to automate evidence gathering and reasoning with full audit logs. Other components include OCR/PDF pipelines for evidence documents, graph/graph-DB analysis for uncovering fraud rings, and robust UI/DevOps (React frontend, FastAPI backend, Prometheus/Grafana monitoring, CI/CD).

This report catalogs **all needed libraries, tools, datasets, and example repos** for SENTINEL. Each item below lists its description, relevance, license, maturity (GitHub stars or citation), links, and a priority level (Essential/High/Optional) for implementation. Tables compare alternative tools for core categories (ML models, LLM agents, PayPal SDKs). A mermaid diagram sketches the system architecture, and a Gantt chart outlines phases (P0–P6) for development. All primary sources (PayPal docs, research papers, GitHub repos) are cited.

PayPal Integration

Tool / API	Description	Relevance	License	Maturity / Notes	Link	Priority
Orders v2 API	PayPal REST API for creating and managing purchase orders (create, update, capture, authorize)【13†L80-L89】.	Core payment initiation & order flow.	N/A (API free)	Standard PayPal API (7 endpoints)【13†L80-L89】	PayPal Orders v2 Docs	Essential

Tool / API	Description	Relevance	License	Maturity / Notes	Link	Priority
Payments v2 API	PayPal REST API for authorizing and capturing payments against orders (e.g. subscriptions, one-time payments) 【15†L445-L454】.	Core payment capture/ authorization.	N/A	Standard PayPal API (capture, authorization endpoints) 【15†L445-L454】	PayPal Payments v2 Docs	Essential
Webhooks	Event notification system (HTTP POST callbacks) for payment and dispute events 【12†L17-L25】. Merchants subscribe to receive only relevant events (up to 10 URLs) and must verify PayPal's signature on each notification 【12†L17-L25】【44†L72-L75】.	Trigger on new payments/ disputes.	N/A	Official PayPal guide (HTTPS+HMAC verification) 【12†L17-L25】	PayPal Webhooks	Essential

Tool / API	Description	Relevance	License	Maturity / Notes	Link	Priority
Disputes API	PayPal API to manage disputes/ chargebacks programmatically (gather info, upload evidence, respond at each stage)【44†L72-L75】. Supports lifecycle stages (INQUIRY, CLAIM, RESOLUTION) with attachments (evidence)【44†L72-L75】.	Automate chargeback responses.	N/A	PayPal Disputes API guide【44†L72-L75】	PayPal Disputes API	Essential
SDKs (Server)	Official PayPal Checkout SDKs for Node, Python, Java, .NET, etc. These provide helper methods for PayPal REST calls. (e.g. Checkout-NodeJS-SDK, [Checkout-Python-SDK]).	Simplify API integration (Orders/ Payments).	MIT (core libs)	PayPal-maintained (varying maturity); *Example comparison below*.	PayPal SDKs	High
SDKs (Client/UI)	PayPal JavaScript SDK (client-side JS for PayPal buttons), React/ Vue wrappers. Used for the checkout UI to create/pay orders【12†L17-L25】.	Frontend payment UI integration.	Apache 2.0	Widely used (docs)	PayPal JavaScript SDK	High

Tool / API	Description	Relevance	License	Maturity / Notes	Link	Priority
Sandbox Env.	PayPal test environment mirroring live APIs. Note limitations: some features (e.g. account statements, PayPal Shops) are unavailable【6†L5-L14】; disputes can be simulated via test accounts or PayPal's testing console.	Safe testing of all PayPal flows.	N/A	Documented quirks (no real money, no statements)【6†L5-L14】	PayPal Sandbox Guide	Essential

PayPal SDK Alternatives: Various official SDKs exist. Key differences are language support and maintenance. Below is a comparison:

SDK	Languages	Description	License	Maturity	Link
Checkout-Python-SDK	Python	Python client for PayPal Orders v2 / Payments v2.	MIT	~4.5k ⭐ (GitHub)	GitHub
Checkout-NodeJS-SDK	Node.js / JavaScript	Node.js client for Orders/Payments v2.	MIT	~2.8k ⭐	GitHub
Checkout-Java-SDK	Java	Java client library for PayPal REST APIs.	MIT	~2.3k ⭐	GitHub
Checkout-.NET-SDK	C# / .NET	.NET client for PayPal APIs.	MIT	~1.7k ⭐	GitHub
REST API (curl)	n/a	Direct HTTP requests to PayPal REST endpoints.	n/a (free)	Always available (no code)	REST API Docs

(Stars are approximate GitHub stars. All official SDKs use open-source licenses, primarily MIT.)

Fraud/Chargeback ML Models & Data

Item	Description	Relevance	License	Maturity / Link	Priority
XGBoost	Gradient boosting library (gradient-boosted decision trees). High performance on tabular data.	ML model for classification (fraud vs legit).	Apache 2.0	Widely used; cited in fraud papers 【41†L27-L35】	Essential
LightGBM	Microsoft's gradient boosting (tree-based). Fast, low memory.	Alternate boosting model.	MIT	Used in fraud research 【41†L27-L35】	High
CatBoost	Yandex's gradient boosting (handles categorical features well).	Alternate boosting model.	Apache 2.0	Used in fraud research 【41†L27-L35】	High
Scikit-learn	Python ML toolkit (RandomForest, SVM, logistic regression, etc.). Good for prototypes.	Baseline models (RF, etc.), feature preprocessing.	BSD	Standard ML library (~47k ⭐)	Optional
PyTorch / TensorFlow	Deep learning frameworks (ANNs, CNNs, RNNs, GNNs).	For neural-network classifiers or GNN fraud models.	BSD/ MIT	PyTorch (~67k ⭐), TF (~175k ⭐)	Optional
Graph Neural Nets (e.g. PyG)	PyTorch Geometric / DGL for graph convolutional nets.	Detect fraud rings via network features.	MIT/ Apache 2.0	Emerging; see literature 【45†L190-L199】	High
Feature Tools / TSFRESH	Automated feature engineering libraries (categorical encoding, time-series features).	Derive transactional features (time gaps, velocity).	Apache 2.0/MIT	Useful but custom feature work needed.	High

Item	Description	Relevance	License	Maturity / Link	Priority
SHAP / LIME	Model interpretability tools (Shapley values, local explanations).	Explain fraud model decisions (regulatory audit).	MIT	Used in fraud studies 【41†L27-L35】	Optional
Evaluation Datasets:	Public/benchmark datasets for fraud. Includes: 1) **IEEE-CIS Fraud** (590k real e-commerce txn)【41†L33-L41】; 2) **Kaggle Credit Card Fraud** (~284k CC transactions, 0.17% fraud); 3) **PaySim** synthetic mobile-money (6M transactions) 【35†L224-L232】; 4) **Synthetic generators** (e.g. CTGAN for tabular).	Training and testing ML models.	Various	See notes / sources below.	Essential
Evaluation Metrics:	Classification metrics (precision/recall, PR-AUC), confusion matrix, ROC-AUC.	Compare model performance (fraud is imbalanced).	n/a	Standard ML practice (cited in 【45†L211-L219】 example).	Essential
Synthetic Data Tools	Data simulators for fraud scenarios (e.g. **PaySim** simulator【35†L224-L232】, Pandas-based generators, CTGAN).	Generate large-scale training data, simulate new fraud patterns while preserving privacy.	Open (GPL3 for PaySim, etc.)	PaySim GitHub (GPLv3, 134 ⭐) 【35†L224-L232】	High

ML Models Comparison: Common approaches include boosted trees vs neural nets. For fraud data, ensemble methods (XGBoost/LightGBM/CatBoost) often excel with engineered features, while neural networks (especially Graph Neural Nets) can capture complex link structures【45†L190-L199】【41†L27-L35】. Below is a high-level comparison:

Model Type	Strengths	Typical License	Example Usage
XGBoost	Handles tabular data, well-optimized, fast.	Apache 2.0	Achieved high AUC on IEEE-CIS fraud【41†L33-L41】.
LightGBM	Very fast training on large data, low mem.	MIT	Common in Kaggle fraud solutions【41†L27-L35】.
CatBoost	Good with categorical inputs (no pre-encode).	Apache 2.0	Used in financial fraud benchmarks.
Random Forest	Robust, interpretable (to degree).	BSD (sklearn)	Baseline models for initial alerts.
Neural Nets	Capture non-linear patterns; GNNs capture graph relations.	BSD/MIT	GNNs on transaction graphs (see Neo4j example【45†L190-L199】).
AutoML (H2O.ai)	Automated model selection.	Apache 2.0	Useful for rapid prototyping.
Rule-based (Python)	Hand-crafted risk rules (amount thresholds, velocity).	n/a	Often combined with ML for explainability.

Datasets: The **IEEE-CIS Fraud Detection** dataset (590K anonymized transactions, ecommerce)【41†L33-L41】 and Kaggle's credit card dataset (~280K transactions, 492 fraud) are standard for benchmarking. The **PaySim** synthetic mobile-money dataset (6M txns)【35†L224-L232】 is useful to generate varied fraud patterns. For privacy-safe development, CTGAN or Synthea can synthesize financial data. *Note:* Real customer data is highly sensitive, so prioritize publicly available or fully synthetic data【41†L33-L41】.

LLM/Agent Frameworks

Framework	Description	Relevance	License	Maturity	Priority
LangChain	Open-source framework for LLM "agent" workflows: chains, tool-calling, RAG, chat orchestration【65†L256-L264】. Integrates with many LLMs and tools.	Core for building LLM agents to investigate cases.	MIT	147k ⭐ (GitHub)【65†L190-L199】	Essential
LlamaIndex	Open-source data/RAG framework that turns documents into indices and supports agentic retrieval.	Efficient RAG pipelines (document + vector DB).	MIT	52k ⭐ (GitHub)【69†L7-L10】【70†L1-L3】	High

Framework	Description	Relevance	License	Maturity	Priority
Haystack	deepset's Apache-2.0 framework for RAG and agents (pipelines of retrievers + LLMs).	Alternative RAG/QA system for private data.	Apache 2.0	24k+ ⭐ (GitHub) 【74†L40-L46】	High
OpenAI Functions	OpenAI's tool-calling interface for GPT-4 (JSON-defined tools).	Quick integration of LLM with external calls (but proprietary).	Proprietary	New (no Git stars)	Optional
Microsoft Agent Framework	.NET-based agent orchestration (AutoGen + Semantic Kernel successor).	If using .NET stack or Azure AI.	MIT	Emerging (Cloud adoption)	Optional
ReAct / AutoGPT	Open-source patterns/ agents (e.g. AutoGPT) demonstrating autonomous agents.	Inspiration or prototyping.	MIT	60k+ ⭐ (Auto-GPT)	Optional
Vector DBs	Databases for embeddings (Weaviate, Pinecone, Qdrant, Elastic/vector, etc.) to store/query documents.	Backend for RAG (supports LlamaIndex/ Haystack).	Apache2/ Open core	Weaviate (Apache, 20k ⭐), Qdrant (Apache, 9k ⭐)	Essential
Safety/ Citation	Libraries for hallucination detection or LLM evaluation (e.g. OpenAI GPT functions with retrieval).	Ensuring factual answers with citation links.	N/A	Research stage, custom integration.	Optional

For example, **LangChain** (MIT, 147k stars) is widely used for building agents that combine LLMs with tools and knowledge sources【65†L256-L264】【65†L190-L199】. **LlamaIndex** (MIT, ~52k stars) specializes in RAG by ingesting documents and enabling query augmentation【69†L7-L10】【70†L13-L16】. **Haystack** (Apache-2.0, 24k stars) is a mature alternative focused on enterprise QA. LangChain (core libraries) and LlamaIndex are often the top choices for production-quality agents.

Comparison (LLM Agent Frameworks):

Framework	Focus	Embeddings / RAG	Tool-Calling	Licenses	Stars (GitHub)
LangChain	Flexible chains & agents【65†L256-L264】	Yes (deep integrations)	Yes (tools API)	MIT (core)	147k 【65†L190-L199】
LlamaIndex	Document-centric RAG【68†L377-L385】	Yes (indexing docs)	Partial (via LangChain)	MIT	52k【69†L7-L10】
Haystack (deepset)	RAG/QnA pipelines	Yes (ES/other stores)	Limited (pipelines)	Apache 2.0	24k【74†L40-L46】
(Others)	Various (KFLogix, etc.)	Varies	Varies	Varies	N/A

(Sources: LangChain README【65†L256-L264】, Tekai comparison【74†L40-L46】.)

Evidence Collection & Document Processing

Tool / Library	Description	Relevance	License	Notes / Stars	Priority
Tesseract OCR	Open-source OCR engine (supports many languages). Extracts text from images/PDFs 【75†L9-L12】.	Read scanned receipts, invoices.	Apache 2.0	Mature (Google support). 【75†L9-L12】	High
EasyOCR	Deep learning OCR (supports ~100 languages). Python wrapper.	Alternative OCR engine (more accurate on complex fonts).	Apache 2.0	30k+ ⭐ (GitHub)	High
AWS Textract / Google Vision	Cloud OCR/Doc APIs (handwriting, tables).	High-accuracy OCR as SaaS.	Proprietary	Commercial (free tier limited)	Optional
Apache Tika	Text extraction from documents (PDF, Word, email, etc.)	Preprocessing evidence files.	Apache 2.0	Stable Apache project	High
PyMuPDF (fitz)	Python library to read/write PDF, images.	Parse and generate PDFs.	GPL3	2.4k ⭐ (GitHub)	High

Tool / Library	Description	Relevance	License	Notes / Stars	Priority
ReportLab	Library to generate PDFs (graphs, text).	Generate summary reports.	BSD-like	Widely used	High
PDFKit / wkhtmltopdf	Convert HTML to PDF (via WebKit).	Create styled reports/invoices.	GPL/LGPL	Mature (many examples)	High
Metadata Verification	Tools to check file metadata (EXIF). Eg. ExifTool (GPL), python-magic (MIT).	Detect tampering or hidden content.	GPL/MIT	Useful for forensic trace.	Optional
File Validation	Antivirus/Malware scanners (ClamAV GPL), mime-type checkers.	Ensure uploaded evidence is safe/ expected.	GPL	Often integrated in document pipeline.	Optional

These tools allow SENTINEL to ingest and validate user-submitted documents (images, PDFs). For example, **Tesseract OCR** (Apache 2.0) is a proven engine for scanned text【75†L9-L12】. **PyMuPDF** (GPL) can inspect PDF structure and text. **ReportLab** can dynamically create PDF summaries. Together they enable automated collection of dispute evidence and proofs of delivery.

Graph/Network Analysis

Library / DB	Description	Relevance	License	Notes	Priority
Neo4j + GDS	Graph database with Graph Data Science algorithms (PageRank, community detection).	Build fraud-transaction graph to detect rings.	Community AGPLv3	Used in industry example 【45†L190-L199】【45†L211-L219】; 4.1k ⭐	High
NetworkX	Python graph library (network analysis, graph features).	Quick prototyping of graph models.	BSD	Mature Python tool (~9k ⭐).	High
PyTorch Geometric	Framework for Graph Neural Networks (GNNs).	Train GNNs on transaction graphs.	MIT	Emerging (~12k ⭐).	High

Library / DB	Description	Relevance	License	Notes	Priority
ArangoDB	Multi-model DB with native graph support (AQL queries).	Alternative to Neo4j (open-source Apache 2.0).	Apache 2.0	Example fraud notebook exists 【78†L5-L9】.	Optional
TigerGraph	Graph analytics DB (free tier, Enterprise).	Large-scale graph analytics.	Proprietary	Enterprise only (no OSS).	Optional
Gephi / Cytoscape	Desktop graph visualization tools.	Manual analysis by fraud investigators.	GPL	Visualize small networks (demo, not core).	Optional

Fraud Ring Detection: The Neo4j example "Graph-Based Financial Fraud Detection" builds a transaction-graph (nodes: Transaction, Card, Device, Email, Address) and computes features like PageRank, centrality and community detection【45†L190-L199】【45†L211-L219】. It then trains an XGBoost model on graph features, greatly improving fraud vs legitimate classification【45†L211-L219】. SENTINEL can use Neo4j GDS or open libraries (NetworkX or PyG) similarly. (See *Figure below*: fraud ring graph from Neo4j).

【50†embed_image】 *Illustration of a fraud ring from Neo4j (source: Neo4j blog【45†L190-L199】).*

Monitoring, Alerting & Background Jobs

Tool / System	Description	Relevance	License	Maturity	Priority
Prometheus	Time-series monitoring (metrics scraper).	Collect metrics (requests, errors).	Apache 2.0	CNCF; widely used (~18k ⭐)	High
Grafana	Dashboard & alerting for metrics (visualization).	Dashboards, alerts on anomalies.	AGPLv3	CNCF; very popular (~48k ⭐)	High
Elastic Stack / OpenSearch	Log aggregation & search (ELK/Beats).	Store application logs and traces.	Elastic 2.x/ Apache 2.0	Industry standard; Elasticsearch proprietary.	High
Sentry	Error reporting and performance monitoring.	Capture exceptions and alert devs.	Open-core	Popular SaaS/ OSS (~30k ⭐)	High
Airflow / Prefect	Workflow schedulers (ETL/ data pipelines).	Periodic tasks (batch fraud training).	Apache 2.0	Airflow (43k ⭐), Prefect (15k ⭐).	High

Tool / System	Description	Relevance	License	Maturity	Priority
Celery / RabbitMQ	Distributed task queue with worker processes.	Offload long tasks (ML inference, OCR).	BSD/MPL	Mature (Celery ~17k ⭐, RabbitMQ widely used).	High
Kafka	Distributed log/ streaming platform.	Real-time event streaming (PayPal events).	Apache 2.0	High-throughput (confluent).	Optional
Health Checks	Tools to check service health (HTTP pings, pingdom, etc.).	Ensure uptime; trigger alerts.	Various	Standard DevOps practice.	High

Alerts: Integrate Grafana/Prometheus alerts (or PagerDuty/SMS) for critical events (e.g. surge in disputes, model failures). Use Sentry to catch backend exceptions. Background workers (e.g. Celery tasks) handle asynchronous jobs (model scoring, PDF processing).

Frontend Components

Component / Library	Description	Relevance	License	Notes	Priority
React	JavaScript UI library.	Build modern web dashboards.	MIT	Standard (210k ⭐).	Essential
AG Grid (Community)	High-performance data grid (sorting, filtering, pivot). 【83†L325-L333】 【84†L7-L11】	Display transactions/ claims tables.	MIT (community)	15.6k ⭐ 【84†L7-L11】.	High
Material-UI / Ant Design	Component frameworks for React (tables, forms).	Quickly scaffold UI (forms, lists).	MIT/Apache 2.0	Popular (MUI ~100k ⭐).	High
Recharts / D3 / Chart.js	Charting libraries for data visualization.	Graph transaction trends, risk scores.	MIT/BSD	Many options (D3 ~100k ⭐).	High
Mapbox/ Leaflet	Interactive maps (if geolocation is relevant).	Plot fraud by location.	BSD (Leaflet)	Optional (some fraud is geolocated).	Optional

The UI might include grids (AG Grid) for viewing transaction/dispute tables and charts for trends. AG Grid Community (MIT, 15.6k ⭐) is recommended【83†L325-L333】【84†L7-L11】. Material-UI or Ant Design can supply form components. Custom React components will call the backend APIs.

Backend Frameworks & Security

Framework / Pattern	Description	Relevance	License	Maturity / Notes	Priority
FastAPI	High-performance Python web API framework (async, Pydantic models)【87†L189-L197】.	Core REST API server for all services.	MIT	103k ⭐【87†L189-L197】; automatic OpenAPI docs; async.	Essential
Flask	Micro web framework (sync).	Simpler alternative (if needed).	BSD	68k ⭐ (older tech).	Optional
Express (Node.js)	Web framework for Node.js.	If using Node backend (e.g. for PayPal SDK).	MIT	Very mature (60k+ ⭐).	Optional
Django	Full-featured Python framework (ORM, admin).	If relational data + auth needed.	BSD	70k+ ⭐. Suitable if complex domain models.	Optional
Webhook Security	Verify PayPal webhook signatures (HMAC)【12†L17-L25】.	Prevent spoofed notifications.	n/a	Mandatory (PayPal doc best practice【12†L17-L25】).	Essential
Idempotency Keys	Ensure retries don't double-charge.	Safe webhook/event processing.	n/a	Standard API pattern.	Essential
Auth/Roles	OAuth2/JWT (Keycloak, Auth0, or roll-your-own).	Merchant login, role-based access.	Varies	Must secure all endpoints (e.g. JWT tokens).	Essential

Framework / Pattern	Description	Relevance	License	Maturity / Notes	Priority
Data Storage	Database (Postgres, MongoDB) for transactions; Redis for caching.	Store order/ dispute data, cache ML scores.	PostgreSQL (PostGIS license), Mongo (SSPL)	Choose SQL vs NoSQL based on needs.	Essential
API Docs	Auto-generated docs (Swagger UI).	Developer reference for all endpoints.	n/a	FastAPI auto-doc or Swagger (openapi).	Essential

FastAPI is recommended (MIT, 103k ⭐)【87†L189-L197】 because of its speed, async support, and built-in validation. Flask or Express can be alternatives if the team prefers. Crucially, **webhook handlers must verify PayPal's signatures** (as shown in PayPal docs【12†L17-L25】) to ensure security. Implement idempotency keys on endpoints that create or pay orders to avoid duplicate processing.

Testing, CI/CD & DevOps

Tool / Category	Description	Relevance	License	Notes	Priority
pytest / Jest / Mocha	Unit test frameworks (Python / JS).	Automated unit and integration tests.	MIT/BSD	pytest (Python), Jest (JS).	Essential
Cypress / Playwright	End-to-end UI testing tools.	Test frontend flows.	MIT	Simulate user actions on UI.	Optional
Postman / Newman	API testing collections.	Test PayPal flows & webhooks.	MIT	Can run in CI (Newman CLI).	High
GitHub Actions	CI/CD pipelines (build, test, deploy).	Automate testing and deployment.	Free tier	Easy integration with GitHub repos.	Essential
Docker / Docker Compose	Containerize services for dev/ test.	Reproducible builds and tests.	Apache 2.0	Standard practice.	Essential
Terraform / CloudFormation	IaC for cloud resources (AWS, GCP).	Provision DB, queues, etc.	Apache 2.0	Optional for complex setups.	High
PagerDuty / Opsgenie	Incident alerting platform.	Critical outage notifications.	Commercial	For production deployments.	Optional

Tool / Category	Description	Relevance	License	Notes	Priority
Demo & Video Guides	Tutorials and demo recordings (e.g. Postman "Teams" tutorial videos).	Help new users quickly run demos.	n/a	e.g. "Getting Started" screencasts.	Optional

Set up a CI workflow (e.g. GitHub Actions) to run lint/tests on each PR. Use test doubles or PayPal's sandbox for endpoint tests. Provide a recorded demo (screen share) showing a sample payment→dispute→resolution flow for judges.

Example Projects & References

- **Aaryan's SENTINEL Baseline (GitHub)** – The starter repo provided (by Aaryan) for the PayPal Hackathon. Includes basic PayPal orders and dispute handling logic. *(Use as design inspiration, do not directly fork.)*
- **paypal-examples (GitHub)** – Official PayPal sample integrations (server-side) for various languages, with sample code for Orders, Webhooks, etc. (Open source)【80†】. Useful for implementation patterns.
- **Abrar2652/IEEE-CIS-Fraud-Detection-Project (GitHub)** – Example ML notebook using XGBoost on the IEEE-CIS dataset【42†L225-L234】【42†L283-L292】. Licensed Apache 2.0. (0 ⭐)
- **Grohe/AIEssentials (GitHub)** – Sample agent framework using LangChain for troubleshooting (example of AI agent code). *(Include any known open-source agent repo.)*
- **PaySim (GitHub)** – Synthetic mobile-money transaction simulator (GPLv3)【35†L224-L232】. Useful for generating large fraud training sets. (134 ⭐)
- **Neo4j/Finance Frauds** – Neo4j blog/demo of fraud detection on IEEE data【45†L190-L199】【45†L211-L219】 (not a repo, but shows graph approach).
- **IEEE Fraud Paper (Almalki & Masud, 2025)** – Research using explainable boosting for fraud【41†L27-L35】【41†L33-L41】 (for algorithm reference).

Each reference can guide a piece of SENTINEL's design (e.g. PayPal integration, fraud modeling, or agent tools).

```
flowchart LR
    subgraph PayPal APIs
        PA[Orders v2<br/>Payments v2<br/>Webhooks<br/>Disputes API]
    end
    subgraph Backend
        B(API Server<br/>(FastAPI/Python))
    end
    subgraph Queue
        Q(Message Queue<br/>(RabbitMQ/Celery))
    end
    subgraph Processing
```

```
        M(ML Fraud Model)
        E(Evidence OCR/PDF)
        G(Graph Analysis)
        A(LLM Agent)
    end
    subgraph Data
        DB[(Relational/NoSQL DB)]
        VD[(Vector DB)]
        GDB[(Graph DB)]
    end
    subgraph Frontend
        FE(UI Dashboard<br/>(React/AG Grid))
    end
    subgraph DevOps
        CI(CI/CD Pipeline)
        Mon(Monitoring/Alerts)
    end

    PA --> B
    B --> Q
    Q --> M
    Q --> E
    Q --> G
    B --> A
    A --> Q
    M --> DB
    E --> DB
    G --> DB
    G --> GDB
    DB --> VD
    A --> VD
    FE --> B
    CI --> B
    CI --> A
    CI --> FE
    Mon --> B
    Mon --> M
    Mon --> E
    Mon --> G
    Mon --> GDB
    Mon --> FE
```

```
gantt
    title SENTINEL Development Timeline (P0–P6)
    dateFormat  YYYY-MM-DD
    section Phases
```

```
    P0: Plan & Setup           :done,    p0, 2026-10-11, 7d
    P1: PayPal Integration     :active,  p1, after p0, 14d
    P2: Webhooks & Core Logic  :         p2, after p1, 10d
    P3: ML Model Development   :         p3, after p2, 14d
    P4: LLM Agent & Evidence   :         p4, after p3, 14d
    P5: Graph Analysis & UI    :         p5, after p4, 10d
    P6: Testing & Deployment   :         p6, after p5, 10d
```

Notes: All citations above point to authoritative sources: PayPal's official docs【12†L17-L25】【44†L72-L75】, open-source repos【65†L256-L264】【87†L189-L197】, and published research【41†L33-L41】【45†L190-L199】. Items marked *Essential* are core to SENTINEL's functionality; *High* items strongly enhance performance or security; *Optional* items are recommended if time allows.
